SEVEN BOT

BAC BO MASTER

Projeto e código-fonte — versão inicial

Este documento reúne os ficheiros do projeto para consulta e cópia. O PDF é uma versão de leitura; para instalar e executar a aplicação, utilize os ficheiros originais do projeto ZIP.

README.md

SEVEN BOT — BAC BO MASTER

Conteúdo

Frontend minimalista, backend Flask e endpoint de análise de imagem via modelo de visão configurado na OpenRouter.

Replit

1. Cria um Repl Python e envia todos estes ficheiros para a raiz.

2. Em Secrets, cria `OPENROUTER_API_KEY` e `VISION_MODEL`.

3. Instala: `pip install -r requirements.txt`

4. Executa: `python app.py`

5. Publica o backend num alojamento compatível com Python.

Configuração

Escolhe na OpenRouter um modelo que aceite imagens e confirma o ID, preço e limites atuais. Não publiques a chave da API.

Limitações

Não existe integração com Elephant Bet, leitura de vídeo em direto, apostas automáticas, conselho de 12 modelos ou supervisor nesta versão. A leitura depende da imagem. Estatísticas históricas não garantem o próximo resultado. A publicação como site estático na Vercel não executa este backend Flask sem adaptação.

index.html

```
<!doctype html><html lang="pt"><head><meta charset="utf-8"><meta name="viewport" content="width=device-
width,initial-scale=1"><title>SEVEN BOT | BAC BO MASTER</title><link rel="stylesheet" href="style.css"></
head><body><main class="shell"><header><div class="logo">7</div><div><h1>SEVEN BOT</h1><small>BAC BO
MASTER</small></div><span class="live">● SISTEMA</span></header><section class="hero"><small>ANÁLISE
VISUAL • ESTATÍSTICA • IA</small><h2>Analisa a mesa.<br><em>Compara os padrões.</em></h2><p>Carrega uma
captura de ecrã para analisar os resultados visíveis.</p></section><section class="panel"><label
class="drop" for="imageInput"><b>■</b><strong id="uploadTitle">CARREGAR IMAGEM DA MESA</strong><small
id="uploadHint">JPG, PNG ou WEBP</small><img id="preview" hidden></label><input id="imageInput"
type="file" accept="image/png,image/jpeg,image/webp" hidden><button id="analyzeBtn" disabled>GERAR
ANÁLISE</button><p class="hint">Confirma sempre a leitura. Resultados incertos não serão inventados.</
p></section><section id="loading" class="panel" hidden>A analisar imagem…</section><section
id="resultPanel" class="panel" hidden><small>RESULTADO</small><h3 id="resultTitle"></h3><p
id="resultSummary"></p><div class="stats"><div><small>Resultados lidos</small><b id="readCount">0</b></
div><div><small>Métodos executados</small><b id="methodCount">0</b></div><div><small>Imagem</small><b
id="imageQuality">—</b></div></div><h4>Sequência identificada</h4><div id="sequence" class="sequence"></
div><details><summary>Ver análise estatística</summary><pre id="details"></pre></details><button
id="resetBtn" class="secondary">NOVA IMAGEM</button></section><aside><b>Nota importante</b><p>Bac Bo é
aleatório. Frequências e padrões históricos não garantem o próximo resultado.</p></aside><footer>SEVEN
BOT • Ferramenta experimental</footer></main><script src="script.js"></script></body></html>
```

style.css

```
*{box-sizing:border-box}body{margin:0;background:radial-gradient(ellipse at 50% 0,#35101b,transparent
42%),#080a10;color:#f5f6fa;font-family:Arial,sans-serif}.shell{max-width:700px;margin:auto;padding:22px
16px}header{display:flex;align-items:center;gap:12px;border-bottom:1px solid #293140;padding-bottom:
18px}.logo{background:linear-gradient(135deg,#ff4568,#94122f);padding:12px 17px;border-radius:12px;font-
size:25px;font-weight:bold}h1{font-size:17px;letter-spacing:2px;margin:0}header small,.hero>small{color:
#ff6682;letter-spacing:2px;font-size:10px}.live{margin-left:auto;color:#8bdcb3;font-size:10px}.hero{
padding:40px 4px 22px}.hero h2{font-size:clamp(32px,8vw,48px);line-height:1.1;margin:12px 0;letter-
spacing:-1px}.hero em{color:#ff5474;font-style:normal}.hero p, .hint{color:#9ba5b5;font-size:13px;line-
height:1.6}.panel{background:#121925;border:1px solid #2a3444;border-radius:17px;padding:18px;margin:
14px 0}.drop{min-height:205px;border:1px dashed #657083;border-radius:12px;background:#0b111a;display:
flex;flex-direction:column;align-items:center;justify-content:center;gap:12px;text-align:center;padding:
15px;cursor:pointer}.drop b{font-size:38px;color:#ff5474}.drop strong{font-size:12px;letter-spacing:1px}.
drop small{font-size:10px;color:#929bab}img{max-width:100%;max-height:250px;object-fit:contain}button{
width:100%;padding:16px;border:0;border-radius:10px;margin-top:14px;background:linear-gradient(90deg,
#f13b5e,#bd183b);color:white;font-weight:bold;letter-spacing:1px}button:disabled{opacity:.4}.secondary{
background:#293446}.hint{text-align:center;font-size:10px}.stats{display:grid;grid-template-columns:
repeat(3,1fr);gap:8px}.stats div{background:#0b111a;padding:12px;border-radius:9px}.stats small{display:
block;color:#9ba5b5;font-size:9px}.stats b{display:block;margin-top:7px;font-size:18px}.sequence{display:
flex;flex-wrap:wrap;gap:6px}.token{padding:8px;border-radius:6px;font-size:11px;font-weight:bold}.J{
background:#153d68;color:#8bc5ff}.B{background:#5b1a2b;color:#ff9caf}.E{background:#594719;color:
#ffe08a}details{margin-top:18px}summary{color:#ff7891;cursor:pointer}pre{white-space:pre-wrap;overflow-
```

```
wrap:anywhere;color:#bac3d1;font-size:11px}aside{margin-top:18px;padding:14px;background:#211b12;border-
left:3px solid #d69b38;border-radius:8px}aside b{color:#ffd27b;font-size:11px}aside p{font-size:10px;
color:#c4b99f;line-height:1.6}footer{text-align:center;color:#788294;font-size:10px;padding:20px}@media(
max-width:420px){.shell{padding:16px 12px}.panel{padding:14px}}
```

script.js

```
const input=document.querySelector('#imageInput'),preview=document.querySelector('#preview'),
btn=document.querySelector('#analyzeBtn');let file=null;input.onchange=()=>{file=input.files[0];if(
!file)return;if(!['image/jpeg','image/png','image/webp'].includes(file.type)||file.size>10*1024*1024){
alert('Escolhe JPG, PNG ou WEBP até 10 MB.');input.value='';return}preview.src=URL.createObjectURL(file);
preview.hidden=false;document.querySelector('#uploadTitle').textContent=file.name;document.querySelector(
'#uploadHint').textContent='Imagem carregada';btn.disabled=false};btn.onclick=async()=>{if(!file)return;
document.querySelector('#loading').hidden=false;document.querySelector('#resultPanel').hidden=true;btn.
disabled=true;try{let fd=new FormData();fd.append('image',file);let r=await fetch('/api/analyze-image',{
method:'POST',body:fd});let d=await r.json();if(!r.ok)throw Error(d.error||'Falha na análise');document.
querySelector('#resultTitle').textContent=d.status;document.querySelector('#resultSummary').
textContent=d.summary;document.querySelector('#readCount').textContent=(d.sequence||[]).length;document.
querySelector('#methodCount').textContent=d.methods_executed||0;document.querySelector('#imageQuality').
textContent=d.image_quality||'—';let s=document.querySelector('#sequence');s.innerHTML='';(d.sequence||[
]).forEach(x=>{let e=document.createElement('span');e.className='token '+x;e.textContent=x;s.append(e)});
document.querySelector('#details').textContent=JSON.stringify(d.analysis||{},null,2);document.
querySelector('#resultPanel').hidden=false}catch(e){alert(e.message)}finally{document.querySelector(
'#loading').hidden=true;btn.disabled=false}};document.querySelector('#resetBtn').onclick=()=>{input.
value='';file=null;preview.hidden=true;btn.disabled=true;document.querySelector('#resultPanel').
hidden=true;document.querySelector('#uploadTitle').textContent='CARREGAR IMAGEM DA MESA';document.
querySelector('#uploadHint').textContent='JPG, PNG ou WEBP'};
```

app.py

```
import os,json,re,base64
from collections import Counter
from flask import Flask,request,jsonify,send_from_directory
import requests
app=Flask(__name__)
app.config['MAX_CONTENT_LENGTH']=10*1024*1024
ALLOWED={'png','jpg','jpeg','webp'}
@app.get('/')
def home(): return send_from_directory('.', 'index.html')
@app.get('/<path:name>')
def assets(name):
    if name in ('style.css','script.js'): return send_from_directory('.',name)
    return jsonify(error='Não encontrado'),404
def stats(s):
    n=len(s); c=Counter(s); out={'total':n,'frequencia':dict(c),'janelas':{str(k):dict(Counter(s[-k:]))
for k in (5,10,20,50)},'alternancias':sum(s[i]!=s[i-1] for i in range(1,n)),'ngrams':{},'transicoes':{},
'intervalos_empate':[],'maior_sequencia':{}}
    for x in 'JBE':
        best=run=0
        for y in s: run=run+1 if y==x else 0; best=max(best,run)
        out['maior_sequencia'][x]=best
    tr={}
    for a,b in zip(s,s[1:]): tr.setdefault(a,Counter())[b]+=1
    out['transicoes']={k:dict(v) for k,v in tr.items()}
    for k in range(2,6): out['ngrams'][str(k)]=dict(Counter(''.join(s[i:i+k]) for i in range(max(0,n-
k+1))).most_common(10))
    pos=[i for i,x in enumerate(s) if x=='E']; out['intervalos_empate']=[pos[i]-pos[i-1] for i in range(
1,len(pos))]
    return out
@app.post('/api/analyze-image')
def analyze():
    f=request.files.get('image')
    if not f or '.' not in f.filename or f.filename.rsplit('.',1)[1].lower() not in ALLOWED: return
jsonify(error='Seleciona uma imagem JPG, PNG ou WEBP.'),400
    key=os.getenv('OPENROUTER_API_KEY'); model=os.getenv('VISION_MODEL')
    if not key or not model: return jsonify(status='IMAGEM RECEBIDA — IA NÃO CONFIGURADA',
summary='Configura OPENROUTER_API_KEY e VISION_MODEL nos Secrets do servidor. Nenhuma sequência foi
inventada.',sequence=[],methods_executed=0,image_quality='Leitura pendente',analysis={'estado':'Modelo
de visão ainda não configurado.'})
    try:
        raw=f.read(); mime=f.mimetype or 'image/jpeg'; url='data:'+mime+';base64,'+base64.b64encode(raw).
decode()
        payload={'model':model,'temperature':0,'messages':[{'role':'system','content':'Lê apenas
resultados Bac Bo claramente visíveis. J=Jogador, B=Banca, E=Empate. Não adivinhes. Responde apenas JSON:
 {"sequence":["J"],"image_quality":"boa|media|fraca","uncertain":true,"notes":"..."}. Se ilegível,
sequence: [].'},{'role':'user','content':[{'type':'text','text':'Extrai os resultados pela ordem visível.
'},{'type':'image_url','image_url':{'url':url}}]}]}
        r=requests.post('https://openrouter.ai/api/v1/chat/completions',headers={'Authorization':'Bearer
'+key,'Content-Type':'application/json'},json=payload,timeout=40);r.raise_for_status()
        text=r.json()['choices'][0]['message']['content']; m=re.search(r'\{.*\}',text,re.S)
        if not m: raise ValueError('JSON não recebido')
        d=json.loads(m.group()); seq=[x for x in d.get('sequence',[]) if x in 'JBE']
        return jsonify(status='ANÁLISE DESCRITIVA' if seq else 'SEM LEITURA CONFIÁVEL',summary=d.get(
'notes','Análise dos elementos legíveis.') if seq else 'Não foi possível confirmar uma sequência. Tenta
uma captura mais nítida.',sequence=seq,methods_executed=25 if seq else 0,image_quality=d.get(
```

```
'image_quality','não avaliada'),analysis=stats(seq))
    except Exception:
        app.logger.exception('Falha na visão')
        return jsonify(error='Falha na análise. Confirma a chave, o modelo de visão e a ligação.'),502
@app.get('/api/health')
def health(): return jsonify(status='ok',vision_configured=bool(os.getenv('OPENROUTER_API_KEY') and os.
getenv('VISION_MODEL')))
if __name__=='__main__': app.run(host='0.0.0.0',port=int(os.getenv('PORT',5000)))
```

requirements.txt

```
Flask>=3.0,<4\nrequests>=2.31,<3\n
```

.env.example

```
OPENROUTER_API_KEY=coloca_a_chave_nos_secrets\nVISION_MODEL=id_de_modelo_com_visao\n
```

.gitignore

```
__pycache__/\n.env\n.venv/\n*.db\nuploads/*\n!uploads/.keep\n
```

uploads/.keep

```
(ficheiro vazio — marcador de pasta)
```